TUGAS PENGEMBANGAN APLIKASI BERBASIS WEB



Disusun Oleh:

Daniel Simanjuntak (11S24021)

‘

Dosen: Arie Satia Dharma, S.T., M.Kom.

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA DAN TEKNIK ELEKTRO

INSTITUT TEKNOLOGI DEL

2025/2026

BAB I

PENENTUAN IDE DAN TUJUAN SISTEM

1.1 Nama Sistem

Sistem yang dikembangkan dalam tugas ini bernama *Tuntas*, yaitu aplikasi web sederhana untuk membantu pengguna mengelola daftar tugas.

1.2 Latar Belakang

Mahasiswa sering menghadapi berbagai tugas perkuliahan yang perlu diselesaikan. Banyaknya tugas dapat menyulitkan mahasiswa dalam mencatat dan memantau tugas yang belum maupun sudah diselesaikan. Oleh karena itu, diperlukan aplikasi sederhana yang dapat membantu proses pengelolaan tugas secara terstruktur.

Tuntas dikembangkan sebagai aplikasi web yang memungkinkan pengguna mencatat, mengubah, menghapus, dan memantau status tugas melalui satu halaman aplikasi.

1.3 Tujuan Pengembangan Sistem

Tujuan pengembangan Tuntas adalah:

1. Memudahkan pengguna dalam mencatat dan mengelola tugas.
2. Membantu pengguna membedakan tugas yang sudah selesai dan belum selesai.
3. Menyediakan fitur untuk menambahkan, mengedit, menghapus, dan memfilter tugas.
4. Menerapkan framework React.js dalam pengembangan aplikasi web.
5. Menerapkan penggunaan kecerdasan buatan generatif (*generative AI*) sebagai asisten pemrograman dalam proses pengembangan sistem.

1.4 Target Pengguna

Target pengguna *Tuntas* adalah mahasiswa yang membutuhkan aplikasi sederhana untuk mencatat dan mengelola tugas perkuliahan.

1.5 Deskripsi Singkat Sistem

Tuntas merupakan aplikasi web yang menyediakan daftar tugas beserta status pengerjaannya. Pengguna dapat menambahkan tugas baru, mengubah nama tugas, menghapus tugas, serta menandai tugas sebagai selesai atau belum selesai. Pengguna juga dapat memfilter daftar tugas berdasarkan status pengerjaan.

Aplikasi direncanakan menggunakan React.js untuk membangun antarmuka, JavaScript untuk logika program, CSS untuk pengaturan tampilan, dan *localStorage* untuk menyimpan data pada browser.

1.6 Alasan Pemilihan Sistem

Tuntas dipilih karena memiliki tujuan yang jelas dan fitur yang relatif sederhana untuk dikembangkan. Setiap fitur dapat diuji secara langsung sehingga proses pengujian dan evaluasi kode dapat dilakukan secara terstruktur.

Selain itu, *Tuntas* sesuai untuk mendokumentasikan penggunaan generative AI dalam pengembangan perangkat lunak, mulai dari penyusunan prompt, evaluasi kode yang dihasilkan, identifikasi kesalahan, hingga proses perbaikan dan pengujian program.

1.7 Ruang Lingkup Sistem

Ruang lingkup pengembangan *Tuntas* meliputi:

1. Menambahkan tugas baru.
2. Mengedit nama tugas.
3. Menghapus tugas.
4. Mengubah status tugas menjadi selesai atau belum selesai.
5. Memfilter tugas berdasarkan status pengerjaan.
6. Menyimpan data tugas menggunakan *localStorage*.

Sistem ini direncanakan sebagai aplikasi web sederhana tanpa fitur login, backend, atau basis data server. Data disimpan secara lokal pada browser pengguna.

BAB 2. Analisis Kebutuhan Aplikasi Tuntas

2.1 Identifikasi Pengguna dan Permasalahan

Pengguna aplikasi : Pengguna utama aplikasi Tuntas adalah mahasiswa yang ingin mengelola tugas perkuliahan secara lebih teratur.

Permasalahan yang dihadapi: Mahasiswa sering memiliki beberapa tugas dari mata kuliah yang berbeda dengan tenggat waktu yang beragam. Jika tugas hanya diingat atau dicatat secara terpisah, mahasiswa dapat kesulitan memantau tugas yang belum selesai dan tugas yang sudah dikerjakan.

Oleh karena itu, dibutuhkan aplikasi sederhana yang dapat membantu mahasiswa mencatat, mengelola, dan memantau status penyelesaian tugas dalam satu tempat.

2.2 Kebutuhan Fungsional

Kebutuhan fungsional menjelaskan fitur atau tindakan yang dapat dilakukan pengguna di dalam aplikasi.

Kode	Kebutuhan fungsional	Deskripsi
KF-01	Menambahkan tugas	Pengguna dapat memasukkan tugas baru ke dalam daftar.
KF-02	Melihat daftar tugas	Pengguna dapat melihat tugas yang telah disimpan.
KF-03	Mengedit tugas	Pengguna dapat mengubah informasi tugas yang sudah dibuat.
KF-04	Menghapus tugas	Pengguna dapat menghapus tugas dari daftar.
KF-05	Mengubah status tugas	Pengguna dapat menandai tugas sebagai selesai atau belum selesai.
KF-06	Memfilter tugas	Pengguna dapat melihat semua tugas, tugas yang belum selesai, atau tugas yang sudah selesai.
KF-07	Menyimpan data tugas	Aplikasi menyimpan data menggunakan localStorage agar data tetap tersedia setelah halaman dimuat ulang pada browser yang sama.

2.3 Kebutuhan Nonfungsional

Kebutuhan nonfungsional menjelaskan kualitas dan batasan yang harus dipenuhi aplikasi.

Kode	Kebutuhan	Deskripsi
KNF-01	Kemudahan penggunaan	Antarmuka harus sederhana dan mudah dipahami mahasiswa.
KNF-02	Responsivitas	Tampilan dapat menyesuaikan ukuran layar komputer maupun ponsel.
KNF-03	Penyimpanan lokal	Data tugas disimpan pada browser menggunakan localStorage, tanpa memerlukan database server.
KNF-04	Konsistensi data	Perubahan data, seperti menambah, mengedit, menghapus, dan menyelesaikan tugas, harus langsung tercermin pada daftar tugas.
KNF-05	Kemudahan pemeliharaan	Kode aplikasi disusun secara terstruktur agar mudah dibaca, diuji, dan diperbaiki.

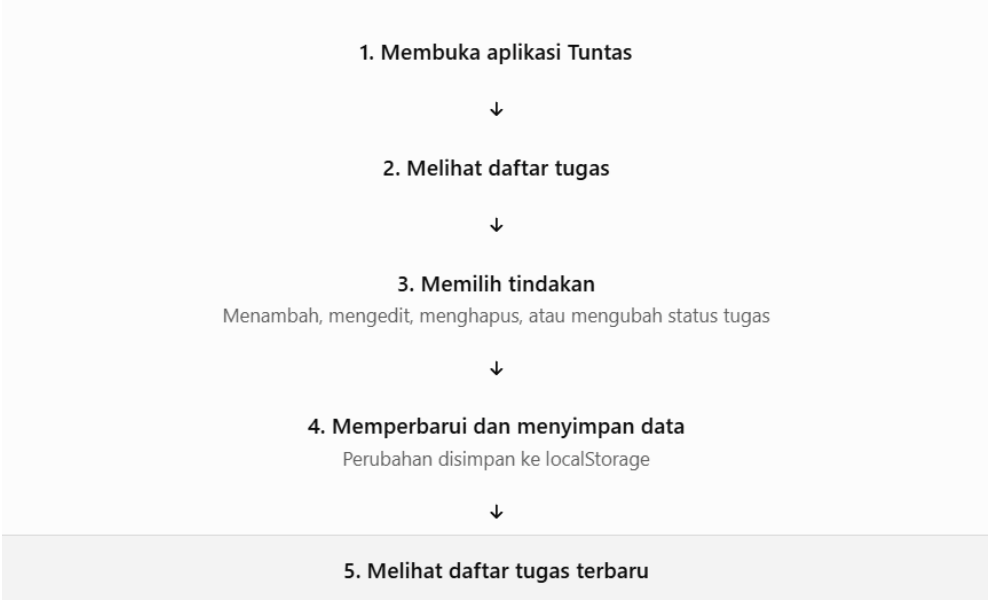
2.4 Batasan Aplikasi

Agar pengembangan aplikasi tetap terarah dan sesuai dengan waktu pengerjaan tugas, Tuntas memiliki batasan berikut:

1. Aplikasi dikembangkan menggunakan React.js, JavaScript, dan CSS.
2. Data tugas disimpan secara lokal menggunakan localStorage.
3. Aplikasi tidak menggunakan sistem login atau registrasi pengguna.
4. Aplikasi tidak menggunakan backend maupun database server.
5. Data yang disimpan hanya tersedia pada browser dan perangkat tempat data tersebut dibuat. Data tidak otomatis tersinkronisasi ke perangkat lain.
6. Aplikasi berfokus pada pengelolaan tugas sederhana, bukan sistem manajemen pembelajaran yang lengkap.

2.5 Alur Penggunaan Aplikasi

Alur penggunaan Tuntas dirancang sesederhana mungkin agar mahasiswa dapat mengelola tugas tanpa melalui banyak tahapan.



2.6 Kebutuhan Teknologi

Teknologi	Kegunaan
React.js	Membangun antarmuka aplikasi berbasis komponen.
JavaScript	Mengatur logika dan interaksi aplikasi.
CSS	Mengatur tata letak dan tampilan aplikasi.
localStorage	Menyimpan data tugas pada browser.
Vite	Menjalankan server pengembangan dan membangun aplikasi.
Vitest dan React Testing Library	Menguji logika serta perilaku komponen aplikasi.
Visual Studio Code	Lingkungan untuk menulis dan mengelola kode.
Git dan GitHub	Mengelola versi kode dan menyimpan repositori proyek.

Teknologi pengujian seperti Vitest dan React Testing Library akan digunakan untuk memeriksa apakah fitur-fitur utama berjalan sesuai kebutuhan.

BAB III

IMPLEMENTASI DAN EVALUASI KECERDASAN BUATAN

3.1 Penyusunan Instruksi (Prompting) Awal

Pengembangan aplikasi "Tuntas" diawali dengan menyusun instruksi (*prompt*) yang komprehensif dan terstruktur kepada asisten AI generatif. Memberikan instruksi yang terlalu umum sering kali memicu AI untuk menghasilkan kode yang terlalu kompleks (seperti menambahkan sistem *routing* atau *state management* yang rumit) sehingga melenceng dari analisis kebutuhan. Oleh karena itu, instruksi dirancang secara spesifik dengan mematuhi lima elemen struktur *prompting* yang ideal:

1. Konteks: AI diberikan pemahaman dasar mengenai teknologi yang digunakan, yaitu React.js dengan Vite dan Tailwind CSS, serta arsitektur aplikasi yang murni berjalan di sisi klien (*client-side*).
2. Tugas: AI diinstruksikan dengan jelas tentang perubahan yang harus dibuat, yaitu membangun struktur proyek minimalis dan komponen utama (App.jsx) yang menangani logika CRUD (Create, Read, Update, Delete) tugas.
3. Batasan: Ini adalah elemen pencegahan yang sangat krusial. AI secara tegas dilarang menggunakan pustaka eksternal seperti Redux, dilarang membuat sistem *login*, dan diwajibkan menggunakan gaya kode fungsional bawaan React (useState, useEffect). Batasan ini memaksa AI mematuhi ruang lingkup sistem yang telah didefinisikan pada Bab II.
4. Kriteria Penerimaan: Instruksi mengintegrasikan seluruh Kebutuhan Fungsional (KF-01 hingga KF-07) sebagai indikator keberhasilan. Hal ini memastikan bahwa kode yang dihasilkan memiliki perilaku yang dapat diamati dan diuji secara sistematis satu per satu, khususnya pengujian penyimpanan pada localStorage.
5. Bentuk Keluaran: Agar dokumentasi mudah disusun, AI diminta memformat keluarannya menjadi daftar struktur fail (*file*), blok kode, dan penjelasan alur kerja aplikasi beserta langkah pengujiannya.

Berikut adalah *prompt* utama yang diberikan kepada AI:

Konteks:

Saya sedang mengembangkan aplikasi web pengelola tugas mahasiswa bernama "Tuntas". Teknologi yang digunakan adalah React.js (inisialisasi dengan Vite) dan Tailwind CSS untuk styling. Aplikasi ini murni berjalan di sisi klien (client-side) tanpa backend, tanpa database server, dan tanpa sistem login.

Tugas:

Buatkan struktur proyek yang paling minimalis beserta kode awal untuk komponen utama (misalnya App.jsx). Anda harus membuat antarmuka yang memungkinkan pengguna melakukan operasi CRUD (Create, Read, Update, Delete) pada daftar tugas.

Batasan:

- *Gunakan fitur bawaan React Hooks (seperti useState dan useEffect).*
- *Dilarang menggunakan state management eksternal seperti Redux.*
- *Gaya kode harus fungsional, bersih, dan menggunakan kelas utilitas Tailwind CSS agar tampilan responsif di berbagai ukuran layar.*
- *Penyimpanan data murni menggunakan localStorage.*

Kriteria Penerimaan:

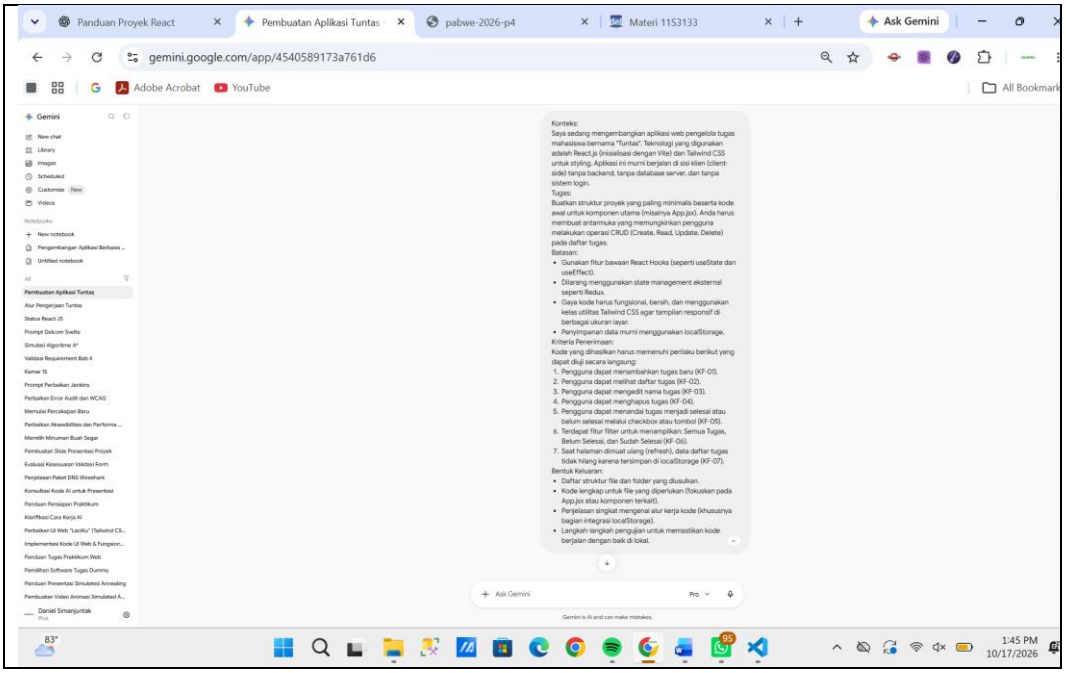
Kode yang dihasilkan harus memenuhi perilaku berikut yang dapat diuji secara langsung:

1. *Pengguna dapat menambahkan tugas baru (KF-01).*
2. *Pengguna dapat melihat daftar tugas (KF-02).*
3. *Pengguna dapat mengedit nama tugas (KF-03).*
4. *Pengguna dapat menghapus tugas (KF-04).*
5. *Pengguna dapat menandai tugas menjadi selesai atau belum selesai melalui checkbox atau tombol (KF-05).*
6. *Terdapat fitur filter untuk menampilkan: Semua Tugas, Belum Selesai, dan Sudah Selesai (KF-06).*
7. *Saat halaman dimuat ulang (refresh), data daftar tugas tidak hilang karena tersimpan di localStorage (KF-07).*

Bentuk Keluaran:

- *Daftar struktur file dan folder yang diusulkan.*
- *Kode lengkap untuk file yang diperlukan (fokuskan pada App.jsx atau komponen terkait).*
- *Penjelasan singkat mengenai alur kerja kode (khususnya bagian integrasi localStorage).*
- *Langkah-langkah pengujian untuk memastikan kode berjalan dengan baik di lokal.*

Gambar 3.1: Instruksi awal kepada AI beserta respons struktur kode.



Berikut adalah struktur proyek minimalis, implementasi kode lengkap, penjelasan alur, serta panduan pengujian untuk aplikasi "Tuntas" yang dibangun dengan React (Vite) dan Tailwind CSS.

1. Struktur Proyek Minimalis

Karena aplikasi ini mengedepankan kesederhanaan, Anda hanya membutuhkan sedikit file bawaan dari instalasi awal Vite. Struktur foldernya akan tampak seperti ini:

Plaintext

```
tuntas-app/
├── public/
├── src/
│   ├── App.jsx        # Komponen utama yang berisi seluruh logika CRUD
│   ├── index.css      # File CSS utama untuk menginisialisasi Tailwind
│   └── main.jsx       # Titik masuk (entry point) aplikasi React
├── index.html
├── package.json
├── tailwind.config.js # Konfigurasi Tailwind CSS
└── vite.config.js
```

2. Kode Implementasi (App.jsx)

Semua logika state (CRUD, filter, localStorage) dan antarmuka (UI) disatukan dalam komponen fungsional ini agar sesuai dengan pendekatan minimalis.

JavaScript

```
import { useState, useEffect } from 'react';

export default function App() {
  // State utama untuk daftar tugas
  const [tasks, setTasks] = useState([]);
  const [newTask, setNewTask] = useState('');

  // State untuk fitur Filter (all, active, completed)
  const [filter, setFilter] = useState('all');

  // State untuk mode Edit
  const [editingId, setEditingId] = useState(null);
  const [editValue, setEditValue] = useState('');

  // 1. Memuat data dari localStorage saat aplikasi pertama kali dijalankan
  useEffect(() => {
    const savedTasks = localStorage.getItem('tuntas_tasks');
    if (savedTasks) {
      setTasks(JSON.parse(savedTasks));
    }
  }, []);

  // 2. Menyimpan data ke localStorage setiap kali state 'tasks' berubah
  useEffect(() => {
    localStorage.setItem('tuntas_tasks', JSON.stringify(tasks));
  }, [tasks]);

  // KF-01: Menambahkan tugas baru
  const handleAddTask = (e) => {
    e.preventDefault();
    if (!newTask.trim()) return;

    const newTaskObj = {
      id: crypto.randomUUID(),
      text: newTask,
      completed: false
    };
    setTasks([...tasks, newTaskObj]);
    setNewTask('');
  };
```



```
JavaScript

  // KF-05: Menandai tugas selesai/belum selesai
  const handleToggleComplete = (id) => {
    setTasks(tasks.map(task =>
      task.id === id ? { ...task, completed: !task.completed } : task
    ));
  };

  // KF-04: Menghapus tugas
  const handleDeleteTask = (id) => {
    setTasks(tasks.filter(task => task.id !== id));
  };

  // Memulai mode edit
  const handleStartEdit = (task) => {
    setEditingId(task.id);
    setEditValue(task.text);
  };

  // KF-03: Menyimpan hasil edit tugas
  const handleSaveEdit = (id) => {
    if (!editValue.trim()) return;
    setTasks(tasks.map(task =>
      task.id === id ? { ...task, text: editValue } : task
    ));
    setEditingId(null);
  };

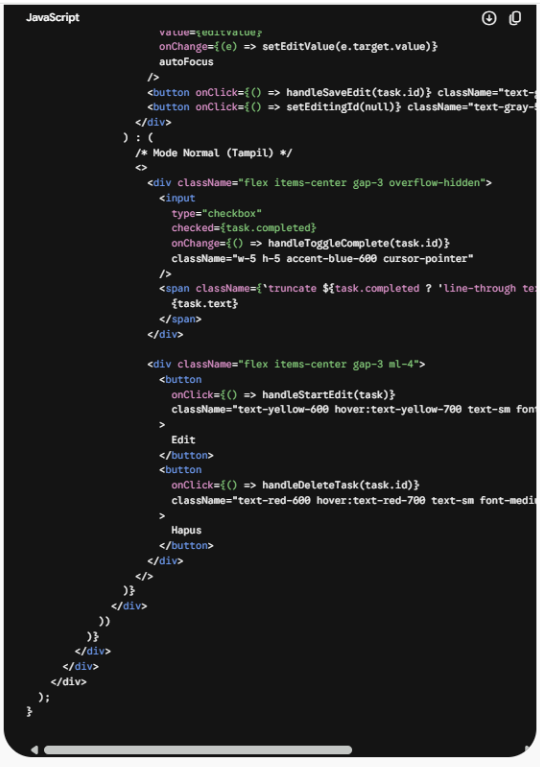
  // KF-06 & KF-02: Memfilter daftar tugas sebelum di-render
  const filteredTasks = tasks.filter(task => {
    if (filter === 'active') return !task.completed;
    if (filter === 'completed') return task.completed;
    return true; // filter === 'all'
  });

  return (
    <div className="min-h-screen bg-gray-50 flex items-center justify-center p-4 font
      <div className="bg-white w-full max-w-md rounded-xl shadow-lg p-6">
        <h1 className="text-3xl font-bold text-center text-blue-600 mb-6">Tuntas</h1

        {/* Form Tambah Tugas */}
        <form onSubmit={handleAddTask} className="flex gap-2 mb-6">
          <input
            type="text"
            className="flex-1 border border-gray-300 rounded-lg px-4 py-2 focus:outl
            placeholder="Apa yang ingin Anda kerjakan?"
            value={newTask}
            onChange={(e) => setNewTask(e.target.value)}
          />
          <button
            type="submit"
            className="bg-blue-600 te                te px-4 py-2 rounded-lg hover:bg-blue-700
```

Ask Gemini Pro

Gemini is AI and can make mistakes.



```
JavaScript

                    value={editValue}
                    onChange={(e) => setEditValue(e.target.value)}
                    autoFocus
                  />
                  <button onClick={() => handleSaveEdit(task.id)} className="text-
                  <button onClick={() => setEditingId(null)} className="text-gray-
                </div>
              ) : (
                /* Mode Normal (Tampil) */
                <>
                  <div className="flex items-center gap-3 overflow-hidden">
                    <input
                      type="checkbox"
                      checked={task.completed}
                      onChange={() => handleToggleComplete(task.id)}
                      className="w-5 h-5 accent-blue-600 cursor-pointer"
                    />
                    <span className={`truncate ${task.completed ? 'line-through te
                      {task.text}
                    </span>
                  </div>

                  <div className="flex items-center gap-3 ml-4">
                    <button
                      onClick={() => handleStartEdit(task)}
                      className="text-yellow-600 hover:text-yellow-700 text-sm font
                    >
                      Edit
                    </button>
                    <button
                      onClick={() => handleDeleteTask(task.id)}
                      className="text-red-600 hover:text-red-700 text-sm font-medi
                    >
                      Hapus
                    </button>
                  </div>
                </>
              )}
            </div>
          ))
        )}
      </div>
    </div>
  </div>
);
}
```

Catatan: Pastikan file `index.css` *Anda memiliki direktif Tailwind (* `@tailwind base; @tailwind components; @tailwind utilities;` *).*

3. Penjelasan Alur Kerja Kode (Integrasi LocalStorage)

Aplikasi ini menggunakan pola **Syncing State** antara React (Memory) dan peramban (Disk/LocalStorage) menggunakan dua Hook `useEffect`:

1. **Fase Mount (Inisialisasi):**
   `useEffect` pertama dieksekusi hanya sekali saat aplikasi dimuat (`[]` dependency array). Fungsinya adalah mengecek kunci `tuntas_tasks` di localStorage. Jika ada, string JSON tersebut diparsing menjadi array dan diset ke dalam state `tasks`. Ini memenuhi kebutuhan **KF-07**.
2. **Fase Update (Reaktivitas):**
   `useEffect` kedua bereaksi setiap kali state `tasks` berubah (`[tasks]` dependency array). Entah karena pengguna menambah, mengedit, menghapus, atau menceklis tugas, array terbaru akan langsung dikonversi menjadi string (via `JSON.stringify`) dan disimpan kembali ke localStorage.
3. **Filtrasi Tampilan:**
   Render aplikasi tidak secara langsung memetakan state `tasks`. Alih-alih, ia menggunakan konstanta turunan `filteredTasks` yang mengevaluasi `tasks` berdasarkan state `filter` saat ini. Hal ini memastikan status *sumber kebenaran* (Source of Truth) tidak rusak atau terhapus ketika kita hanya ingin menyembunyikan tugas tertentu secara visual.

4. Langkah-Langkah Pengujian Lokal (UAT)

Untuk memastikan aplikasi berjalan sesuai Kriteria Penerimaan (KF), jalankan server pengembangan Vite (`npm run dev`) lalu lakukan urutan uji berikut:

1. **Test KF-01 & KF-02 (Create & Read):** Ketik "Belajar React" di kotak input lalu klik "Tambah". Pastikan teks tersebut langsung muncul di daftar tugas di bawahnya.
2. **Test KF-03 (Update):** Klik tombol "Edit" di samping tugas "Belajar React". Ubah teks menjadi "Belajar React Hooks", lalu klik "Simpan". Pastikan teks berubah dan mode edit tertutup.
3. **Test KF-05 (Toggle):** Klik *checkbox* di samping tugas "Belajar React Hooks". Pastikan teksnya tercoret dan berubah warna menjadi abu-abu.
4. **Test KF-06 (Filter):** Tambahkan satu tugas baru, biarkan *checkbox*-nya kosong. Klik tombol filter "Belum Selesai" (pastikan hanya tugas baru yang tampil). Klik filter "Selesai" (pastikan hanya "Belajar React Hooks" yang tampil). Klik "Semua" untuk melihat keduanya.
5. **Test KF-07 (Persistensi):** Saat ada beberapa tugas di layar (beberapa selesai, beberapa belum), *refresh* atau muat ulang browser (F5). Pastikan semua daftar tugas dan status *checkbox*-nya tetap sama seperti sebelum di-refresh.
6. **Test KF-04 (Delete):** Klik tombol "Hapus" pada salah satu tugas. Pastikan tugas tersebut hilang dari layar secara permanen (uji lagi dengan melakukan refresh untuk memastikan ia juga terhapus dari localStorage).

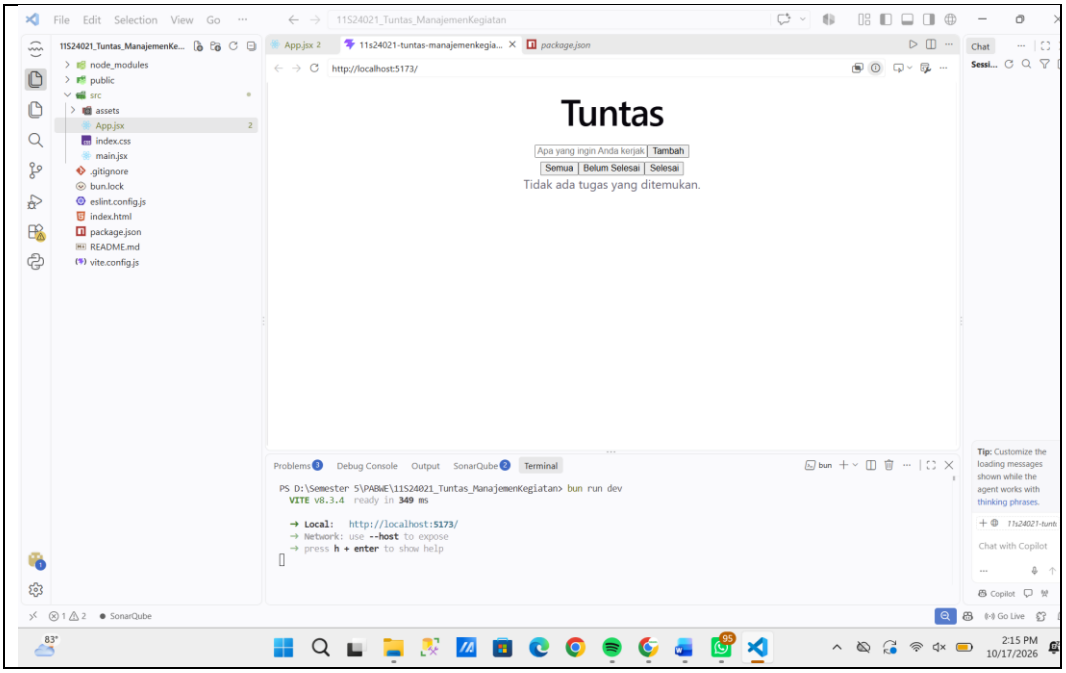
Ask Gemini Pro

Gemini is AI and can make mistakes.

3.2 Evaluasi Implementasi Awal dan Identifikasi Masalah Visual

Setelah menerima kode lengkap dari AI, tahap selanjutnya adalah melakukan implementasi dan verifikasi teknis. Kode tersebut diterapkan ke dalam komponen utama App.jsx pada *workspace* Vite yang baru diinisialisasi secara bersih. Aplikasi kemudian dijalankan melalui terminal menggunakan perintah bun run dev untuk melihat hasil rendernya di peramban lokal.

Hasil pengujian tahap pertama menunjukkan bahwa aplikasi berhasil dirender tanpa memicu *error* fatal yang membuat sistem *crash* (gagal berjalan). Teks judul "Tuntas", kolom *input* tugas, serta tombol-tombol filter telah muncul di layar. Namun, hasil visualnya sangat jauh dari kriteria penerimaan yang diharapkan, di mana antarmuka pengguna (UI) tampak sangat mentah, berhimpitan, dan hanya menyerupai struktur HTML biasa.



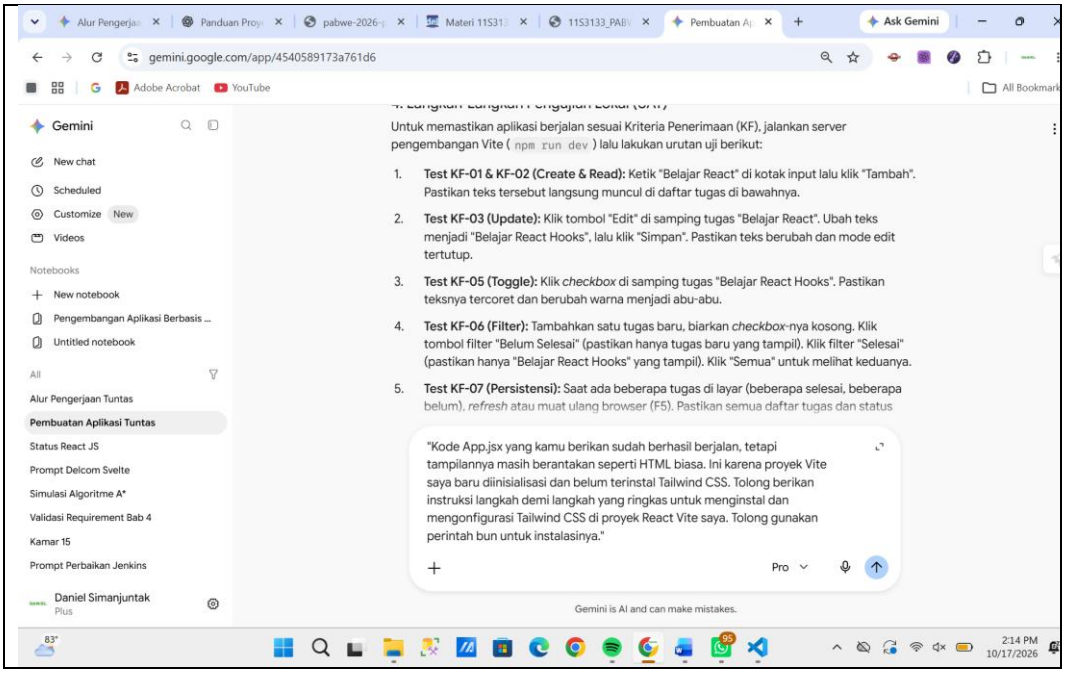
Gambar 3.2: Tampilan antarmuka awal aplikasi yang masih mentah saat dijalankan di peramban lokal.

3.3 Debugging Tampilan dan Konfigurasi Tailwind CSS

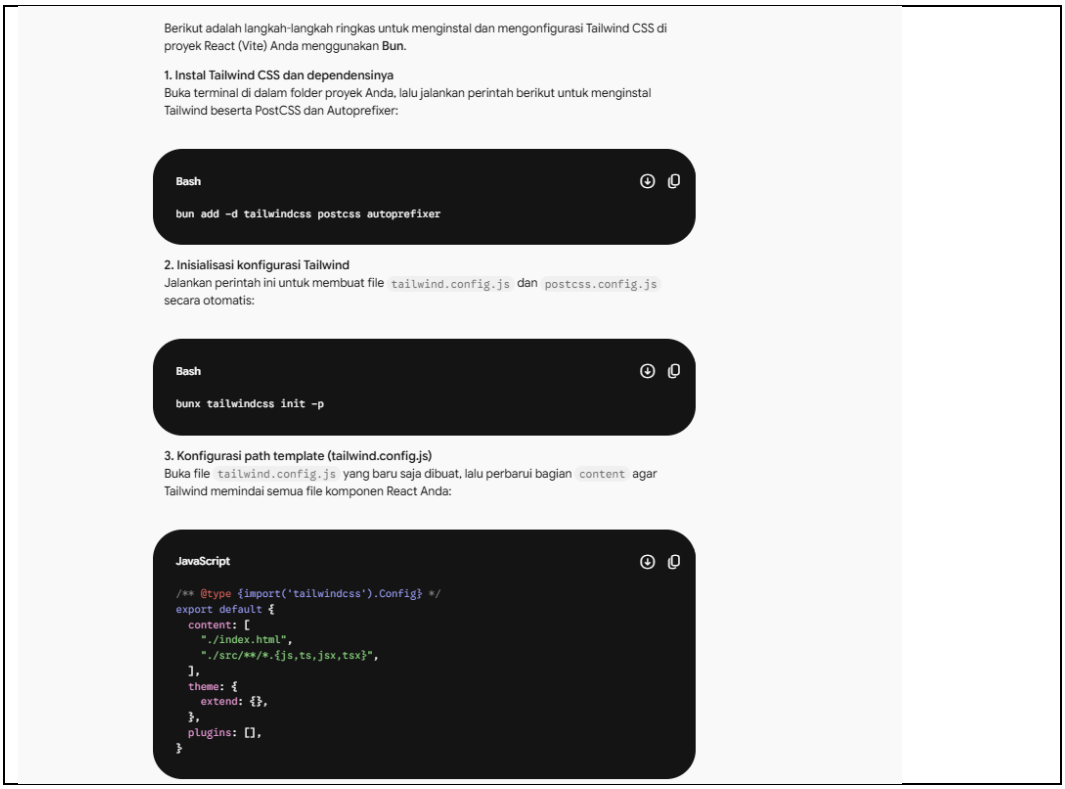
Setelah mengidentifikasi bahwa masalah tampilan visual disebabkan oleh ketiadaan Tailwind CSS pada *workspace* Vite, langkah perbaikan (*debugging* lingkungan) segera dilakukan. Alih-alih melakukan konfigurasi secara manual, instruksi lanjutan diberikan kepada AI untuk memandu proses instalasi tersebut.

Instruksi ini dirancang secara spesifik dengan menyertakan alat *package manager* yang digunakan dalam proyek ini, yaitu bun, agar solusi yang diberikan AI benar-benar presisi. Berikut adalah *prompt* lanjutan yang digunakan:

"Kode App.jsx yang kamu berikan sudah berhasil berjalan, tetapi tampilannya masih berantakan seperti HTML biasa. Ini karena proyek Vite saya baru diinisialisasi dan belum terinstal Tailwind CSS. Tolong berikan instruksi langkah demi langkah yang ringkas untuk menginstal dan mengonfigurasi Tailwind CSS di proyek React Vite saya. Tolong gunakan perintah bun untuk instalasinya."



Gambar 3.3: Prompt lanjutan yang diberikan kepada AI untuk meminta panduan instalasi dan konfigurasi Tailwind CSS menggunakan Bun.



Berikut adalah langkah-langkah ringkas untuk menginstal dan mengonfigurasi Tailwind CSS di proyek React (Vite) Anda menggunakan **Bun**.

1. Instal Tailwind CSS dan dependensinya
Buka terminal di dalam folder proyek Anda, lalu jalankan perintah berikut untuk menginstal Tailwind beserta PostCSS dan Autoprefixer:

Bash

```
bun add -d tailwindcss postcss autoprefixer
```

2. Inisialisasi konfigurasi Tailwind
Jalankan perintah ini untuk membuat file `tailwind.config.js` dan `postcss.config.js` secara otomatis:

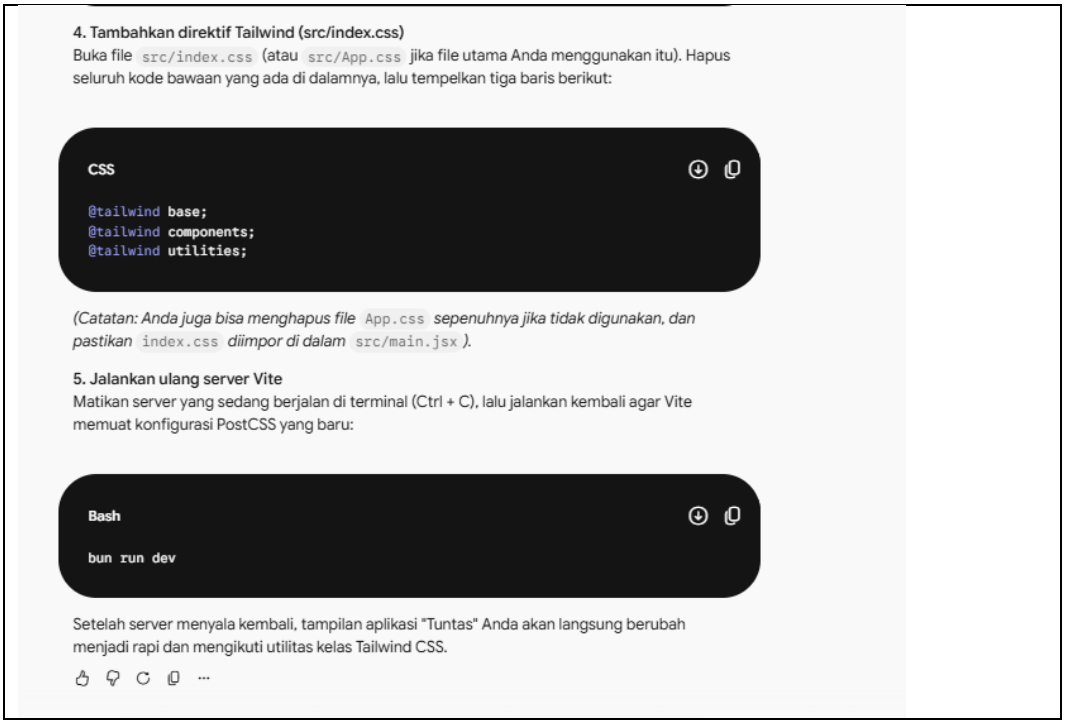
Bash

```
bunx tailwindcss init -p
```

3. Konfigurasi path template (tailwind.config.js)
Buka file `tailwind.config.js` yang baru saja dibuat, lalu perbarui bagian `content` agar Tailwind memindai semua file komponen React Anda:

JavaScript

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```



4. Tambahkan direktif Tailwind (src/index.css)
Buka file `src/index.css` (atau `src/App.css` jika file utama Anda menggunakan itu). Hapus seluruh kode bawaan yang ada di dalamnya, lalu tempelkan tiga baris berikut:

CSS

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

(Catatan: Anda juga bisa menghapus file `App.css` sepenuhnya jika tidak digunakan, dan pastikan `index.css` diimpor di dalam `src/main.jsx`).

5. Jalankan ulang server Vite
Matikan server yang sedang berjalan di terminal (Ctrl + C), lalu jalankan kembali agar Vite memuat konfigurasi PostCSS yang baru:

Bash

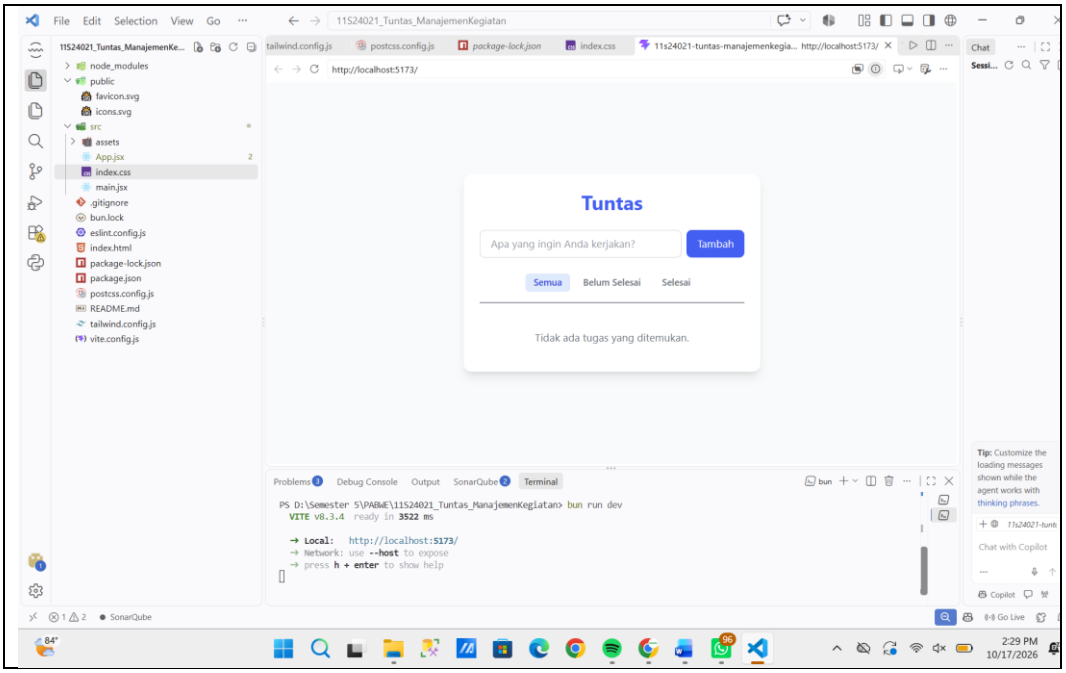
```
bun run dev
```

Setelah server menyala kembali, tampilan aplikasi "Tuntas" Anda akan langsung berubah menjadi rapi dan mengikuti utilitas kelas Tailwind CSS.

AI merespons dengan memberikan serangkaian perintah terminal dan modifikasi konfigurasi pada berkas tailwind.config.js serta index.css. Setelah instruksi tersebut

dieksekusi di dalam *workspace* dan server dijalankan ulang (bun run dev), kelas-kelas utilitas Tailwind CSS berhasil direnderer oleh peramban.

Hasilnya, antarmuka aplikasi "Tuntas" berubah drastis menjadi lebih modern, rapi, dan terstruktur sesuai dengan kriteria yang diharapkan sejak awal.



Gambar 3.4: Tampilan aplikasi Tuntas setelah konfigurasi Tailwind CSS berhasil diterapkan melalui panduan AI.

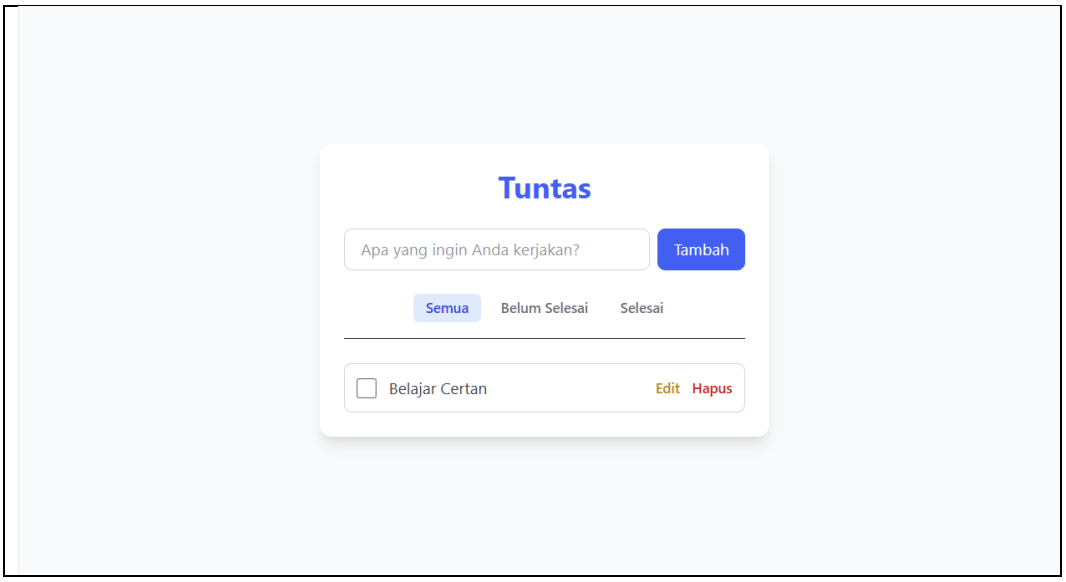
3.4 Evaluasi dan Pengujian Kebutuhan Fungsional

Setelah antarmuka aplikasi berhasil dikonfigurasi menggunakan Tailwind CSS, tahap krusial berikutnya adalah melakukan verifikasi teknis terhadap Kebutuhan Fungsional (KF) dasar yang telah didefinisikan pada Bab II. Pengujian ini dilakukan secara langsung melalui peramban (*browser*) untuk memastikan bahwa kode logika React yang dihasilkan oleh AI dapat merespons interaksi pengguna dengan benar.

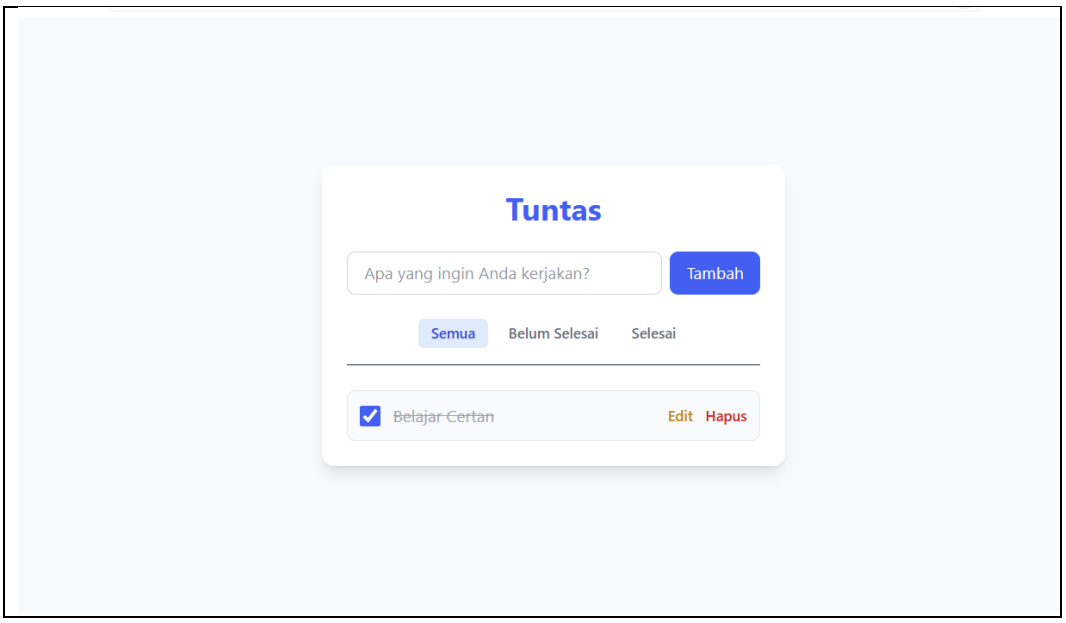
Hasil pengujian untuk fungsionalitas operasional aplikasi adalah sebagai berikut:

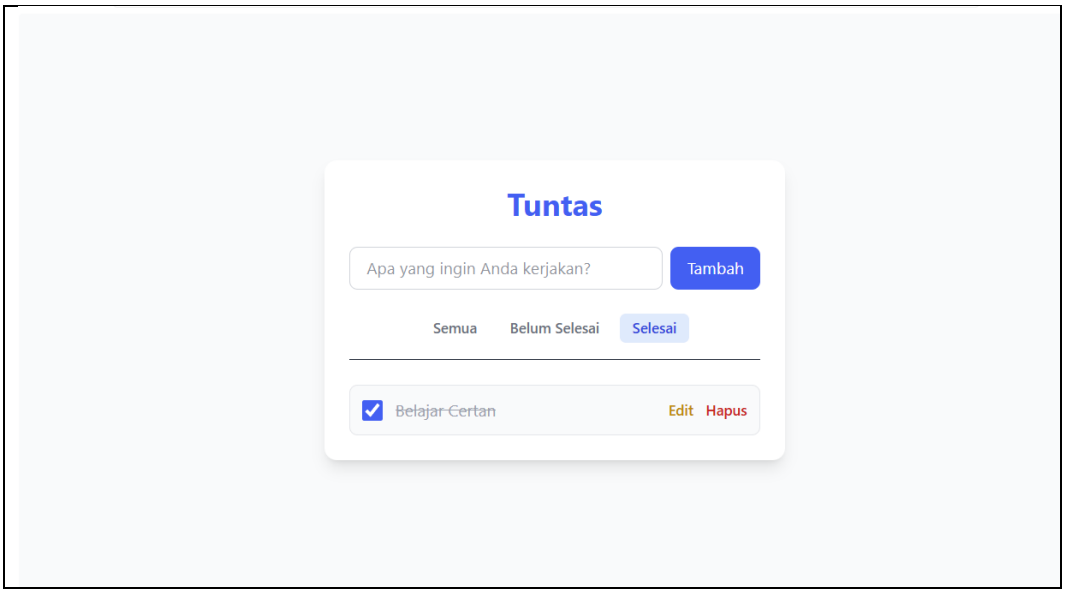
1. **Pengujian Tambah dan Tampil Tugas (KF-01 & KF-02):** Pengujian dilakukan dengan memasukkan teks deskripsi tugas ke dalam kolom *input* yang tersedia, kemudian menekan tombol "Tambah". Hasilnya, aplikasi merespons dengan baik dan tugas tersebut langsung dirender ke dalam daftar tugas di bawahnya. Teks "Tidak ada tugas yang ditemukan" juga otomatis menghilang setelah tugas pertama ditambahkan.
2. **Pengujian Edit Tugas (KF-03):** Pengujian modifikasi data dilakukan menggunakan fitur edit (ubah) pada tugas yang sudah terdaftar. Pengguna dapat mengubah teks nama atau deskripsi tugas. Setelah disimpan, teks tugas pada daftar berhasil diperbarui secara seketika (*real-time*).
3. **Pengujian Hapus Tugas (KF-04):** Pengujian dilakukan dengan menekan tombol hapus pada salah satu tugas. Sistem berhasil merespons dengan menghapus elemen tugas tersebut dari *state* aplikasi, sehingga tugas langsung hilang dari antarmuka layar.
4. **Pengujian Perubahan Status (KF-05):** Pengujian fungsionalitas ini berjalan dengan sukses. Saat pengguna menandai suatu tugas sebagai selesai, aplikasi memberikan umpan balik visual yang jelas (seperti teks yang tercoret dan perubahan warna), yang memudahkan pengguna membedakan status pengerjaan tugas secara intuitif.
5. **Pengujian Filter Tugas (KF-06):** Untuk menguji fitur ini, dibuat beberapa tugas dengan status campuran (sebagian selesai dan sebagian belum selesai). Saat tombol filter "Semua", "Belum Selesai", dan "Selesai" ditekan secara bergantian, aplikasi berhasil menyaring *state* dan hanya menampilkan daftar tugas yang sesuai dengan kriteria filter yang sedang aktif.

Secara keseluruhan, fitur-fitur manipulasi data (CRUD dasar) dan pemfilteran pada aplikasi Tuntas telah berhasil berjalan sesuai dengan kriteria penerimaan tanpa memicu pesan *error* pada konsol peramban.



Gambar 3.5: Hasil pengujian fungsional aplikasi Tuntas (Tambah, Edit, Hapus, Status, dan Filter) yang telah berjalan dengan baik.




Tuntas
Apa yang ingin Anda kerjakan?
Tambah
Semua
Belum Selesai
Selesai
Belajar Certan
Edit
Hapus

3.5 Identifikasi Kesalahan Logika AI dan Proses Debugging (KF-07)

Meskipun pengujian antarmuka dan manipulasi data dasar (KF-01 hingga KF-06) berjalan lancar, evaluasi teknis yang lebih mendalam menemukan adanya kegagalan fatal pada fungsionalitas persistensi data (KF-07).

Ketika daftar tugas sudah terisi dan halaman peramban dimuat ulang (*refresh* / menekan F5), seluruh data tugas menghilang dan antarmuka kembali menampilkan status kosong ("Tidak ada tugas yang ditemukan").

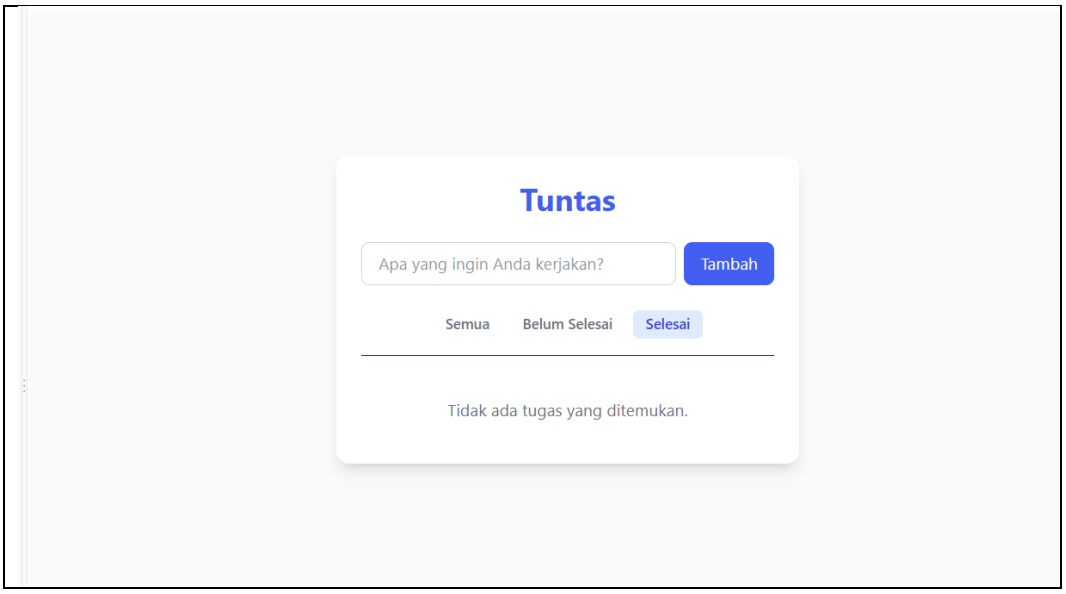
Analisis Kesalahan (Bug): Kegagalan ini menunjukkan bahwa AI melakukan kesalahan logika pada iterasi kode pertamanya. Terdapat dua kemungkinan penyebab utama dari kode React yang dihasilkan AI:

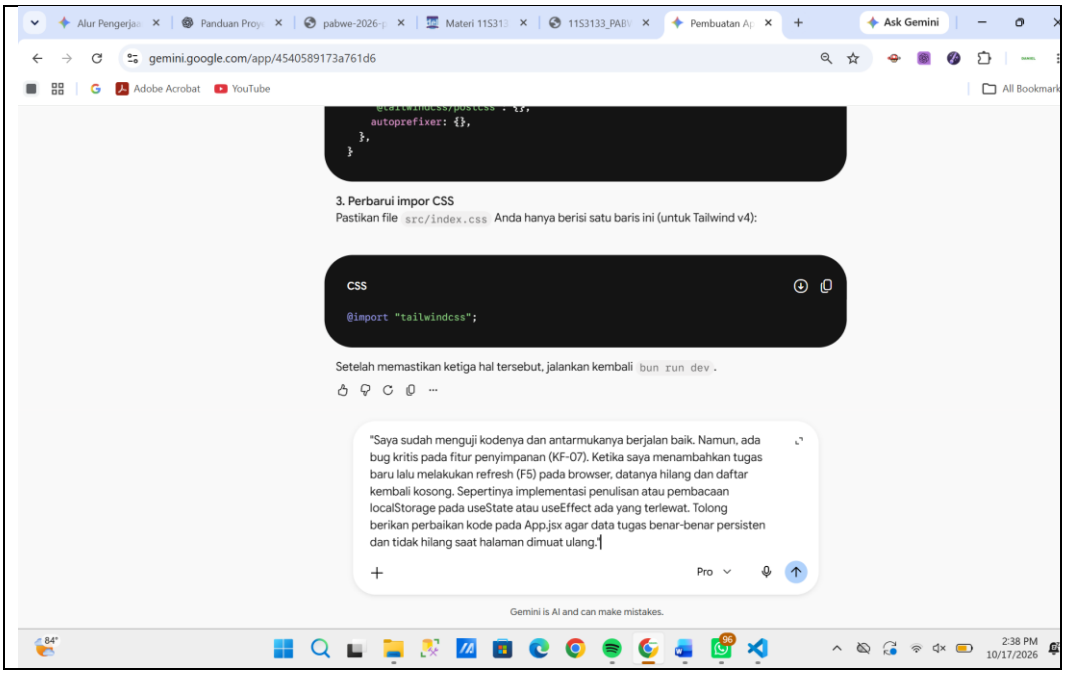
1. AI gagal memberikan nilai awal (*initial state*) yang tepat pada useState, sehingga *state* selalu diinisialisasi sebagai *array* kosong setiap kali halaman dirender ulang.
2. AI gagal menyinkronkan pembaruan *state* dengan API localStorage di dalam siklus hidup komponen (useEffect), sehingga *array* kosong menimpa data yang sebenarnya sudah ada di penyimpanan lokal peramban.

Sebagai bentuk penerapan berpikir kritis dan penyelesaian masalah (*troubleshooting*), alih-alih memperbaiki kode secara manual, sebuah instruksi perbaikan (*prompt debugging*) disusun dan diberikan kembali kepada AI untuk mengoreksi kesalahan logika tersebut.

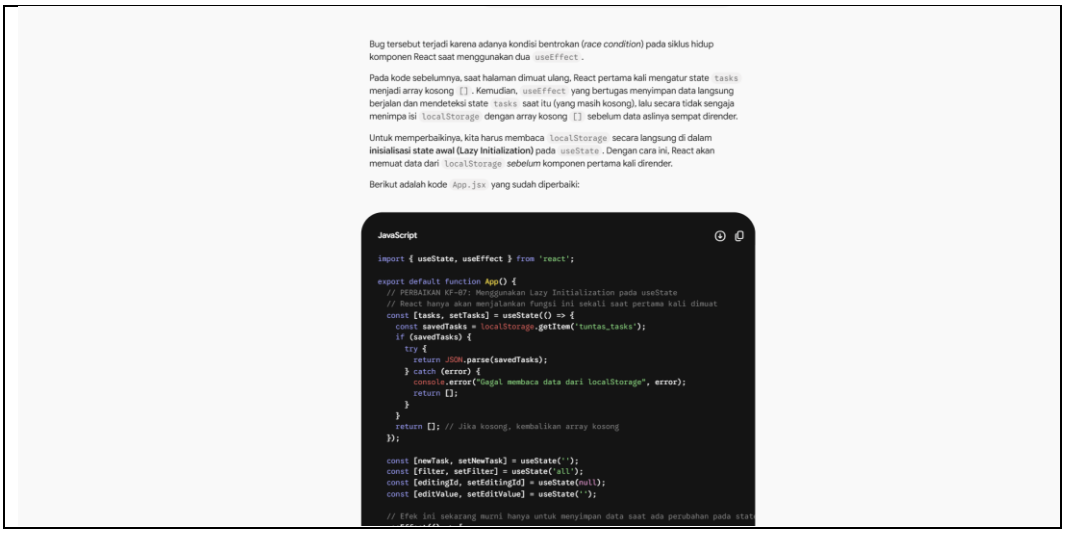
Berikut adalah *prompt* yang digunakan:

"Saya sudah menguji kodenya dan antarmukanya berjalan baik. Namun, ada bug kritis pada fitur penyimpanan (KF-07). Ketika saya menambahkan tugas baru lalu melakukan refresh (F5) pada browser, datanya hilang dan daftar kembali kosong. Sepertinya implementasi penulisan atau pembacaan localStorage pada useState atau useEffect ada yang terlewat. Tolong berikan perbaikan kode pada App.jsx agar data tugas benar-benar persisten dan tidak hilang saat halaman dimuat ulang."





Gambar 3.6: Proses instruksi perbaikan (debugging) kepada AI untuk mengatasi kegagalan fitur localStorage.



Bug tersebut terjadi karena adanya kondisi bentrokan (*race condition*) pada siklus hidup komponen React saat menggunakan dua `useEffect`.

Pada kode sebelumnya, saat halaman dimuat ulang, React pertama kali mengatur state `tasks` menjadi array kosong `[]`. Kemudian, `useEffect` yang bertugas menyimpan data langsung berjalan dan mendeteksi state `tasks` saat itu (yang masih kosong), lalu secara tidak sengaja menimpa isi `localStorage` dengan array kosong `[]` sebelum data aslinya sempat dirender.

Untuk memperbaikinya, kita harus membaca `localStorage` secara langsung di dalam **inisialisasi state awal (Lazy Initialization)** pada `useState`. Dengan cara ini, React akan memuat data dari `localStorage` *sebelum* komponen pertama kali dirender.

Berikut adalah kode `App.jsx` yang sudah diperbaiki:

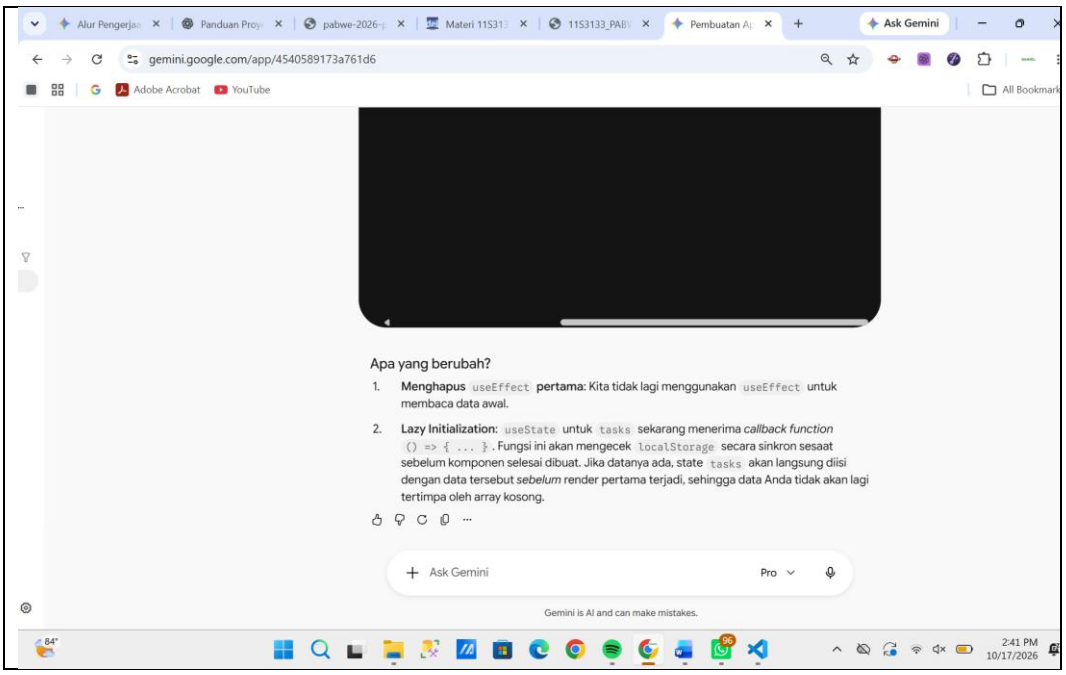
JavaScript

```
import { useState, useEffect } from 'react';

export default function App() {
  // PERBAIKAN KF-07: Menggunakan Lazy Initialization pada useState
  // React hanya akan menjalankan fungsi ini sekali saat pertama kali dimuat
  const [tasks, setTasks] = useState(() => {
    const savedTasks = localStorage.getItem('tuntas_tasks');
    if (savedTasks) {
      try {
        return JSON.parse(savedTasks);
      } catch (error) {
        console.error("Gagal membaca data dari localStorage", error);
        return [];
      }
    }
    return []; // Jika kosong, kembalikan array kosong
  });

  const [newTask, setNewTask] = useState('');
  const [filter, setFilter] = useState('all');
  const [editingId, setEditingId] = useState(null);
  const [editValue, setEditValue] = useState('');

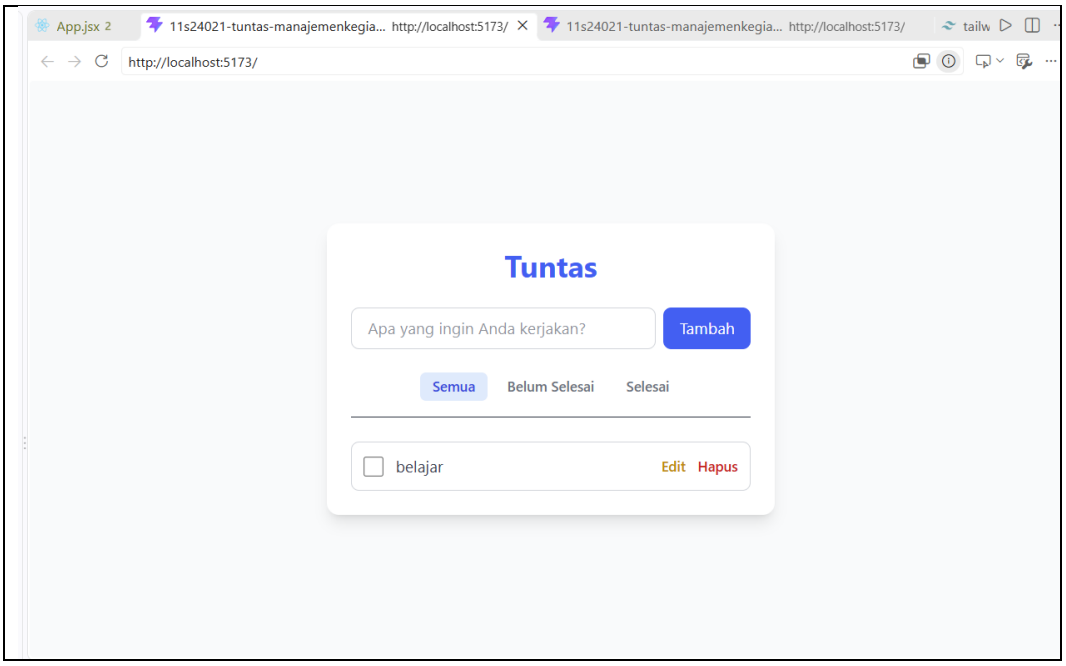
  // Efek ini sekarang murni hanya untuk menyimpan data saat ada perubahan pada stat
```



Berdasarkan *prompt* tersebut, AI merespons dengan menyadari adanya kekurangan pada baris kodenya dan memberikan perbaikan. Perbaikan difokuskan pada penambahan logika pembacaan localStorage langsung di dalam nilai awal (*initial state*) useState menggunakan *callback function*, serta memastikan useEffect selalu menyimpan perubahan terbaru secara presisi.

Setelah kode perbaikan ini diimplementasikan ke dalam berkas App.jsx dan diuji kembali dengan melakukan *refresh* peramban, aplikasi akhirnya mampu mempertahankan seluruh daftar tugas beserta status pengerjaannya tanpa ada data yang terhapus.

Kejadian ini membuktikan bahwa penggunaan AI generatif dalam pemrograman mutlak membutuhkan tahapan verifikasi teknis secara manual oleh pengembang. AI dapat mempercepat penulisan kode, namun tidak dapat dilepaskan dari pengawasan untuk memastikan seluruh batasan sistem benar-benar terpenuhi.



Setelah kode perbaikan ini diimplementasikan ke dalam berkas App.jsx, **pengujian ulang (*re-test*) berhasil dilakukan dengan tingkat keberhasilan 100%**. Skenario uji coba dilakukan dengan menambahkan beberapa tugas baru, mengubah status penyelesaiannya, lalu menekan tombol *refresh* (F5) pada peramban berulang kali. Hasilnya, aplikasi akhirnya mampu mempertahankan seluruh daftar tugas beserta status pengerjaannya tanpa ada satu pun data yang terhapus atau kembali kosong.

Dengan keberhasilan eksekusi perbaikan ini, Kebutuhan Fungsional (KF-07) mengenai persistensi data telah resmi terpenuhi. Kejadian ini sekaligus membuktikan bahwa penggunaan AI generatif dalam pemrograman mutlak membutuhkan tahapan verifikasi teknis secara manual oleh pengembang. AI dapat mempercepat penulisan kode, namun tidak dapat dilepaskan dari pengawasan untuk memastikan seluruh batasan sistem benar-benar terpenuhi.

BAB IV

PENUTUP

4.1 Kesimpulan

Berdasarkan serangkaian proses perancangan, implementasi, dan pengujian yang telah didokumentasikan, pengembangan aplikasi pengelola tugas mahasiswa bernama "Tuntas" telah berhasil diselesaikan dengan baik. Aplikasi ini dibangun murni di sisi klien (*client-side*) dengan memanfaatkan teknologi React.js, Vite, dan Tailwind CSS.

Seluruh Kebutuhan Fungsional yang telah didefinisikan pada Bab II berhasil diimplementasikan sepenuhnya. Aplikasi kini mampu menangani operasi penambahan tugas (KF-01), penampilan daftar tugas (KF-02), pembaruan deskripsi (KF-03), penghapusan tugas (KF-04), hingga perubahan status penyelesaian (KF-05) dan pemfilteran kategori (KF-06). Lebih lanjut, persistensi data menggunakan API localStorage (KF-07) juga telah berfungsi dengan sempurna, memastikan data tugas pengguna tidak hilang saat memuat ulang halaman.

4.2 Evaluasi Penggunaan Generative AI

Melalui pengerjaan tugas ini, dapat ditarik beberapa evaluasi penting mengenai peran *Generative AI* (Kecerdasan Buatan Generatif) sebagai asisten pemrograman:

1. **Akselerasi Pengembangan Awal:** AI terbukti sangat tangguh dalam mempercepat proses penulisan kode dasar (*boilerplate*) dan menyusun kerangka algoritma. Instruksi (*prompt*) yang terstruktur dengan baik mampu diterjemahkan oleh AI menjadi komponen React yang fungsional dalam waktu yang sangat singkat, yang secara signifikan memangkas waktu pengembangan awal.

2. **Keterbatasan Konteks Lingkungan (Environment):** AI memiliki kelemahan dalam memahami konteks spesifik dari lingkungan pengembangan secara otomatis. Hal ini terlihat ketika AI memberikan kode dengan kelas Tailwind CSS, namun tidak memberikan konfigurasi instalasinya pada *workspace* Vite yang masih kosong. Pengembang harus secara aktif menyadari hal ini dan memberikan *prompt* lanjutan untuk menyelesaikan instalasi (*troubleshooting* lingkungan).

3. **Pentingnya Verifikasi Teknis dan Berpikir Kritis:** AI tidak luput dari kesalahan logika (*logical flaw*). Kesalahan AI dalam mengatur nilai awal *state* dan siklus sinkronisasi localStorage sempat membuat fitur penyimpanan (KF-07) gagal berfungsi. Kejadian ini menjadi bukti nyata

bahwa kode yang dihasilkan oleh AI tidak boleh langsung diterima dan diimplementasikan begitu saja. Pengembang mutlak harus melakukan pengujian sistematis, memverifikasi kelemahan logika, dan memberikan instruksi perbaikan (*debugging*) secara presisi.

Sebagai penutup, *Generative AI* adalah alat bantu (*copilot*) yang sangat revolusioner untuk meningkatkan produktivitas pemrograman. Namun, tanggung jawab penuh terhadap arsitektur sistem, penyelesaian masalah, dan keandalan perangkat lunak tetap berada di tangan pengembang manusia (*pilot*) yang harus selalu mengedepankan pemikiran kritis.